

Tarron Smith
03/17/2026

Market Rival

Overview

- Market Rival is a 2D Market simulation game where the player operates a shop competing against an AI rival.
- Each round, AI customers choose which shop to enter and which item(s) to purchase.
- Customer decisions are influenced by price, quality, and other behavioral preferences.
- The player must make strategic purchasing decisions for their inventory to outperform the rival and capture more market demand.

Technology Stack

Language: Java

Framework: LibGDX

Build System: Gradle

IDE: Eclipse

Version Control: Git / GitHub

Core Gameplay Loop

Each round consist of two phases.

Buy Phase:

- Player and rival AI purchase inventory using available cash from wallet
- Inventory purchased becomes available during the sell phase
- Market statistics from previous rounds help guide decisions

Sell Phase:

- Customers enter the market
- Each customer chooses a shop
- Customers evaluate available items and attempt to purchase one
- Shops earn revenue based on successful sale

Goal: Generate more profit than the rival shop by round max (10).



Customer Behaviour System

Customers are modeled with generated behavioral profiles.

Each customer profile is generated using a **stat budget system** that distributes preference weights across several traits.

Customer preferences include:

- **Price** - tendency to prefer cheaper items
- **Quality** - tendency to prefer higher quality products
- **Hype** - tendency to be influenced by items that are currently popular in the market
- **Loyalty** - tendency to return to a shop visited in previous rounds
- **Stockout Aversion** - tendency to avoid shops that failed to satisfy them previously

Customer profiles are created using the **CustomerProfileGenerator**, which:

- assigns each trait a base value
- distributes remaining stat points randomly across the traits
- normalizes the values into a **0.0–1.0 range**

This produces **unique customer personalities** while keeping the system balanced.

Rival AI

Process:

1. Starts with starter mix for inventory since there's no prior data.
2. When there is prior data, scores each item:
 - a. $\text{score} = \text{demandShare} * \text{marginScore}$
 - b. $\text{demandShare} = \text{remainingDemand} / \text{totalDemand}$
 - c. $\text{marginScore} = \text{itemPrice} / \text{vendorCost}$

If an item is popular but gives poor return, scores low. If an item is profitable but nobody wanted it, its score drops.

3. Buys the item with the highest score once.
4. Reduces demand for that item.
5. Recalculates the next best item.

System Architecture

UI Layer

Responsibility:

Handles rendering and player interaction.

Classes:

- **FirstScreen** - Main game screen that coordinates rendering and player input
- **HudRenderer** - Displays statistics such as cash, inventory, and turn data
- **CustomerListRenderer** - Draws the visual customer agents in the market

Simulation Layer

Responsibility:

Controls the market simulation.

Classes:

- **MarketEngine** - Core controller that runs the simulation loop and coordinates systems
- **RoundManager** - Manages phase transitions (Buy → Sell → Results) and round statistics
- **CustomerSpawner** - Generates customers and assigns their destinations in the market
- **MarketPlan** - Determines customer distribution and market demand for each round
- **CustomerProfileGenerator** - Creates randomized customer preference profiles
- **RivalAI** - Determines the rival shop's purchasing and stocking decisions

Model Layer

Responsibility:

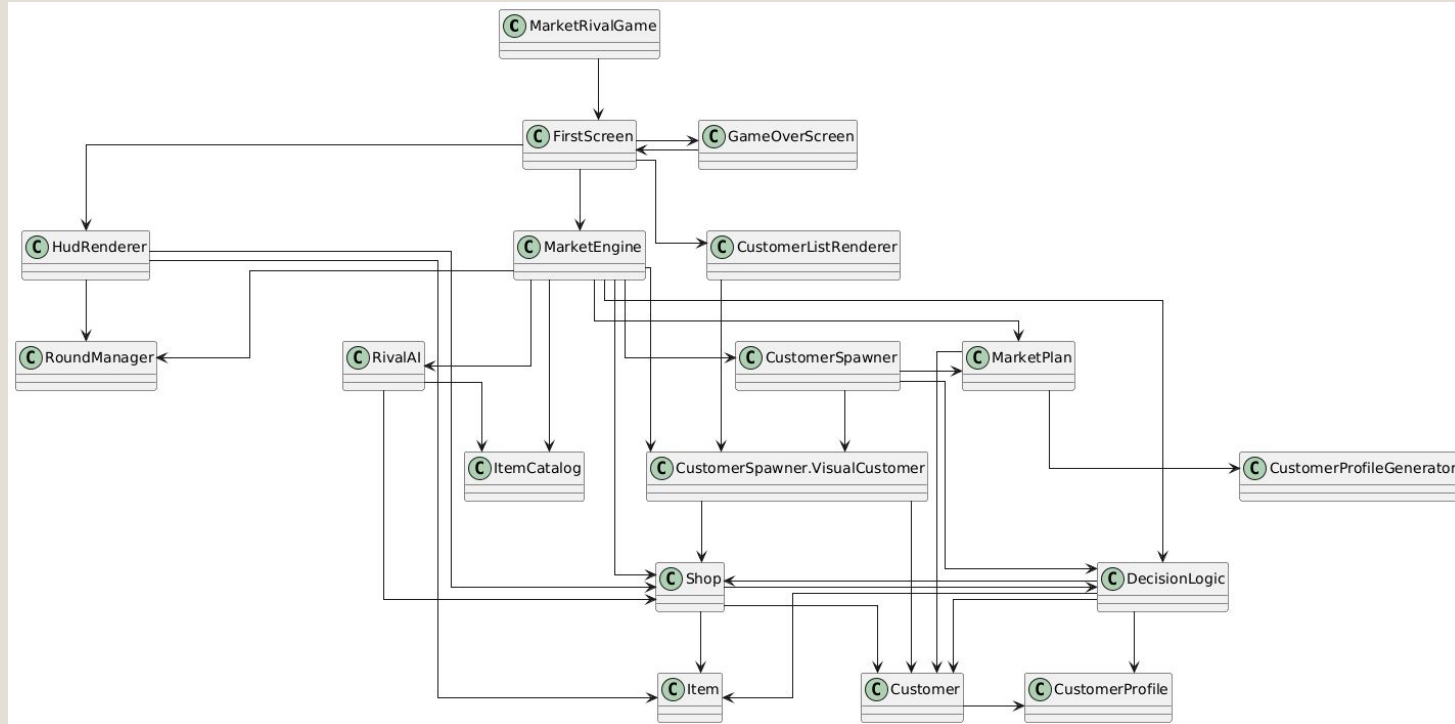
Represents the entities within the market.

Classes:

- **Customer** — Represents an individual shopper with a wallet and preference profile
- **CustomerProfile** - Stores preference weights (price, quality, hype, loyalty, stockout aversion)
- **DecisionLogic** - Determines shop selection and item choice based on customer preferences
- **Shop** - Manages inventory, sales transactions, and revenue tracking
- **Item** - Defines product attributes such as price and quality

System Architecture Diagram

The UI sends input into the engine, the engine coordinates the simulation systems, and those systems operate on the model objects. The results are then sent back to the UI for rendering.



Development Focus

Balancing Random Profile Generation:

Designing a stat-budget system that produces diverse customer personalities while keeping values normalized and preventing extreme or unrealistic behavior.

Designing Preference-Based Decision Logic:

Implementing a scoring system that evaluates shops and items based on customer preferences rather than random selection.

Feature Roadmap

Improved Customer Movement

- Implement proper path-based movement (possibly tile-based)

Dynamic Shop Switching

- Allow customers to leave a shop and visit the rival if desired items are unavailable
- Integrate this with stockout aversion behavior

Shop Attention Mechanics

- Add purchasable attention boosters (signs, advertisements, etc.) that increase the probability that customers choose the player's shop

Item Categories

- Introduce item types (Tech, Food, Utilities, etc.) that allow customers to prefer or search for specific item categories

Expanded Market Behavior

- Improve demand modeling so different item categories fluctuate in popularity



Feedback?

